

UNITED INTERNATIONAL UNIVERSITY

Department of Computer Science and Engineering

PROJECT REPORT

Smart Cloud POS

A Cloud-Based Multi-Store Point of Sale System

with Face Attendance, Salary, Customer Loyalty and AI Assistants

Course Software Engineering Laboratory

Section _____

Submitted to _____

Submission date _____

Submitted by

SL	Name	Student ID
1	Meherab Hossain	011221369
2	Shahinur Jahan Bedhu	011213066
3	Samiul Hyi Shad	0112310317
4	Rohan Hasan	0112230148
5	Muhammad	0112310247

Table of Contents

1. Introduction.....	1
1.1 Short Description.....	1
1.2 Objectives.....	1
1.3 Motivation.....	1
2. Project Description.....	2
2.1 List of Features.....	2
2.2 Description of Each Feature.....	3
3. Technologies Used.....	9
3.1 System Architecture.....	9
3.2 Technology Stack.....	9
3.3 Testing.....	10
3.4 Version Control and Deployment.....	11
4. Future Works.....	12
4.1 Problems We Faced and How We Solved Them.....	12
4.2 Upgrades and New Features.....	13
5. Conclusion.....	14
A. Appendix: Running the Project Locally.....	15

List of figures

Figure 1 – 12: application screenshots (Section 2.2) · Figure 13: system architecture (Section 3.1)

1. Introduction

1.1 Short Description

Smart Cloud POS is a web application we built to run the daily work of a shop, or a chain of shops, from one place: billing, stock, stores, staff and customers. The frontend is written in React, the backend in Node.js with Express, and the data lives in TiDB Cloud, a MySQL-compatible cloud database. The live system is deployed on Vercel (frontend) and Render (backend), so it opens in any browser without installing anything.

Every staff member signs in with a role (Admin, Manager, Cashier or Store Keeper) and sees only the pages that role needs. Customers get a separate portal where they can find the cheapest store for a product, check their loyalty points and ask an AI assistant about offers.

1.2 Objectives

When we started, we wrote down what the finished system had to do. These stayed our checklist for the whole semester:

1. Give cashiers a fast POS screen with barcode scanning, discount, tax and a printed receipt.
2. Keep stock correct in every store, with a full stock in/out history and damaged-goods tracking.
3. Let one Admin manage several stores, while each staff member stays locked to their own store.
4. Record staff attendance with face recognition and calculate salary automatically from it.
5. Reward customers with loyalty points and let them compare prices across stores.
6. Use AI to answer business questions for managers and product questions for customers.
7. Keep the system secure with role-based access, token login and checks on the server side.

1.3 Motivation

A lot of small and medium shops in Bangladesh still write sales in a notebook, or use a separate desktop program in each branch. Stock goes out of sync, the owner cannot see all branches together, and attendance and salary are counted by hand at the end of the month. We saw this ourselves in shops near our homes.

We wanted to build one affordable cloud system that works in any browser, covers the full shop workflow (sell, stock, staff, pay, reward), and adds features like face attendance and AI help that usually only large chains can pay for. It was also a good chance for us to work on a real multi-module project as a team, with Git, testing and deployment, instead of a single-page assignment.

2. Project Description

In total the system has 16 features. Some are used by all four staff roles, some only by the Admin, and one whole part (the customer portal) is only for shoppers. Table 1 lists them, and Section 2.2 explains each one with a screenshot from the running app.

2.1 List of Features

#	Feature	Used by
1	Secure login, roles and first-login setup	All staff
2	Dashboard	Admin
3	Product management and barcode printing	Admin, Manager, Store Keeper
4	POS / sales billing with receipt	Admin, Cashier
5	Sales history and invoices	Admin, Cashier
6	Inventory (stock in / out) and history	Admin, Manager, Store Keeper
7	Damaged / spoiled goods	Admin, Manager, Store Keeper
8	Multi-store management	Admin
9	Workers management (role, store, face)	Admin
10	Face-recognition attendance camera	Admin, Manager
11	Attendance and monthly summary	Admin, Manager
12	Salary calculation and payout	Admin
13	Expenses and suppliers (due ledger)	Admin, Manager
14	Reports	Admin, Manager
15	AI Insights and AI chat assistant	Admin, Manager
16	Customer portal with loyalty points and customer AI	Customers

Table 1: Features of Smart Cloud POS and the roles that use them

2.2 Description of Each Feature

1. Secure login, roles and first-login setup.

Staff sign in with email and password and receive a JWT token. The sidebar shows only the pages their role allows, and every API checks the role again on the server. On the first login a staff member registers their face once and chooses a salary account (Cash, Bank, bKash, Nagad or Rocket). Wrong passwords are limited to 10 tries in 15 minutes to stop guessing.

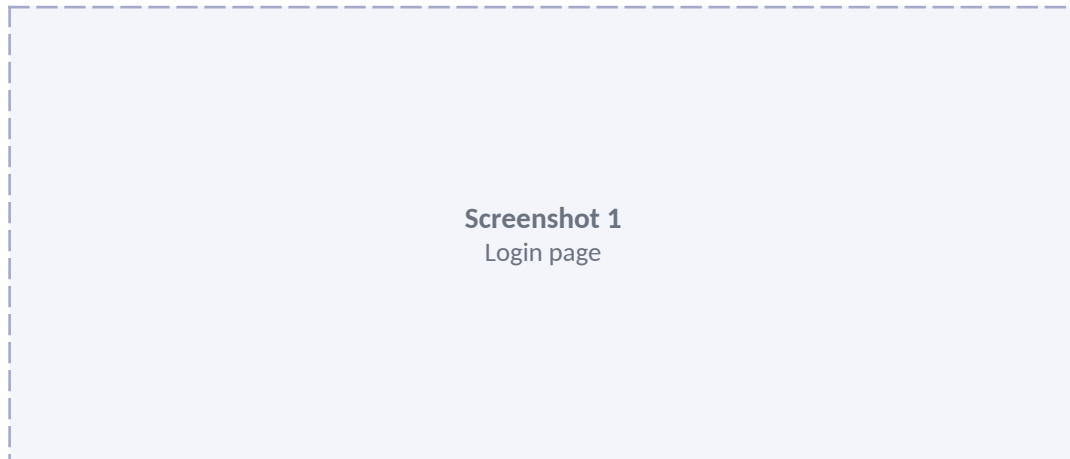


Figure 1: Login page

2. Dashboard.

The Admin sees total sales, today's sales and profit, inventory value, low-stock and out-of-stock counts, a 7-day sales trend, a payment method summary, the top products and a per-store breakdown, all on one screen.

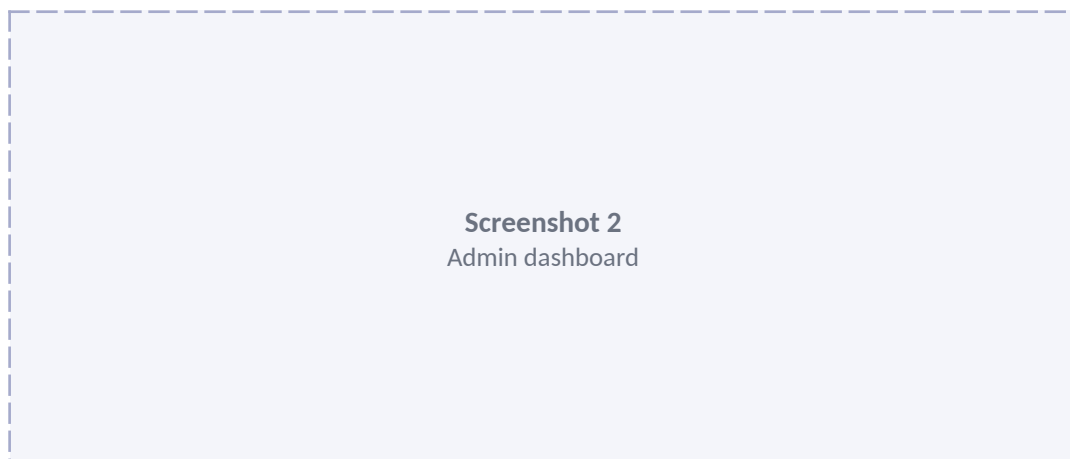


Figure 2: Admin dashboard

3. Product management and barcode printing.

Products are added per store with barcode, category, cost price, selling price, discount percentage and stock. Price and stock can be edited inline, low stock is highlighted in colour, and barcode labels can be printed in any quantity for the shelf.

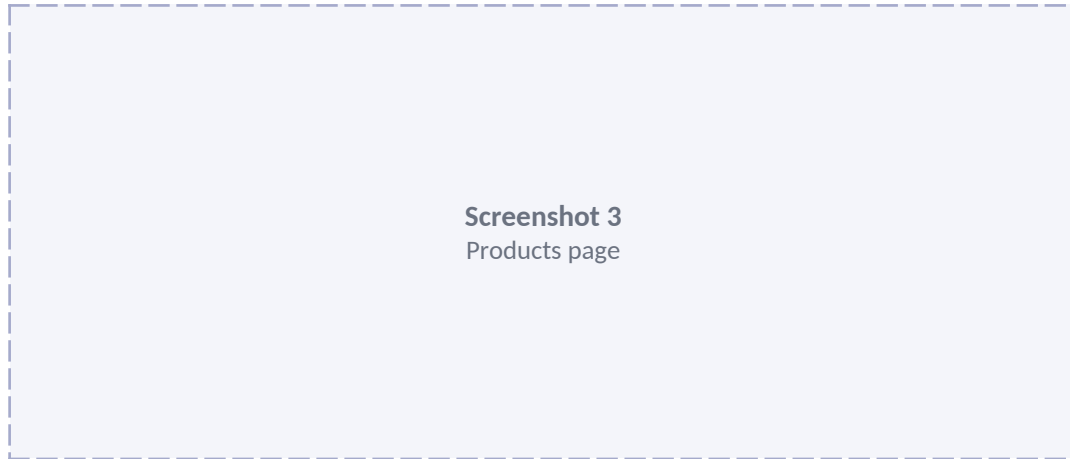


Figure 3: Products page

4. POS / sales billing.

The cashier adds items with a USB or phone barcode scanner, the camera, or a name search. The bill shows subtotal, discount (0–100%, default 0), tax (default 5%, editable), loyalty point redemption and the change to return. The server recalculates every price, checks stock, blocks a cash payment that is less than the total, and only lets staff sell from their own store. A sale can be put on hold and resumed, and a receipt is printed at the end.

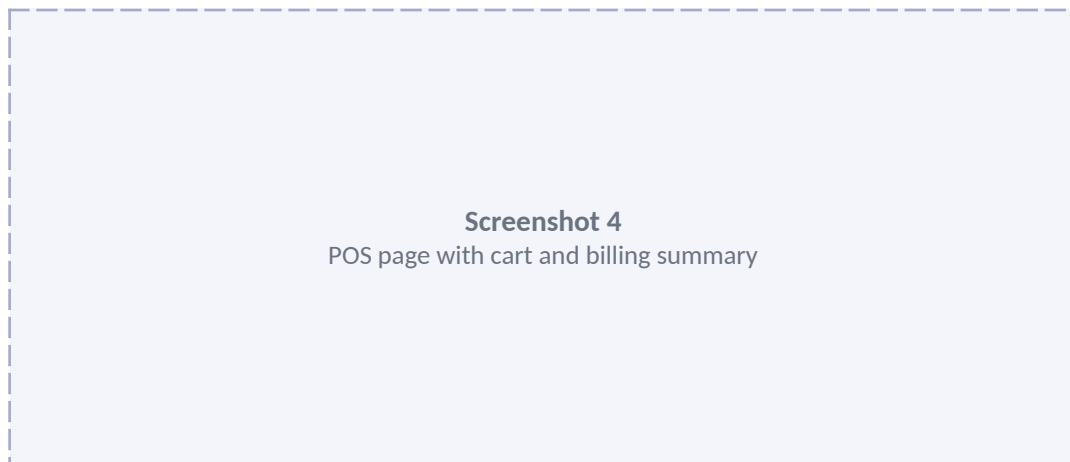


Figure 4: POS page with cart and billing summary

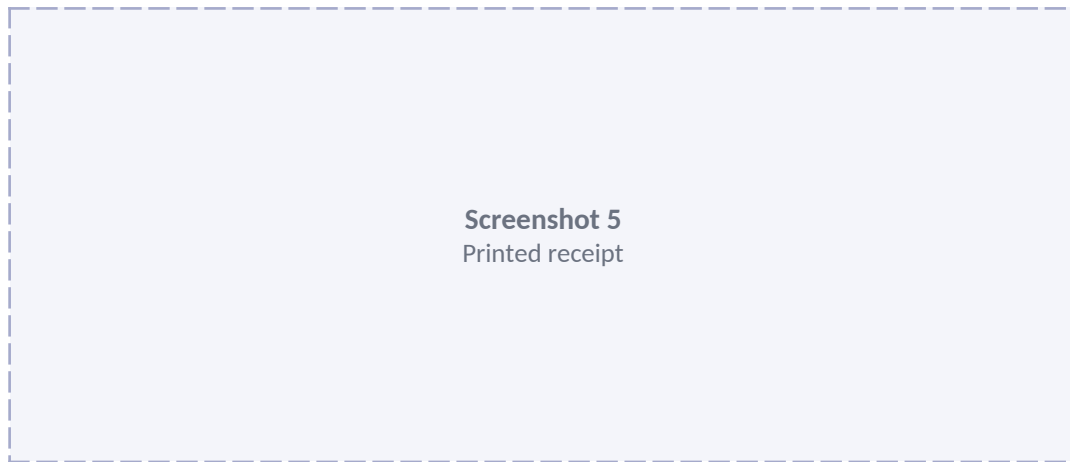


Figure 5: Printed receipt

5. Sales history and invoices.

Every sale is listed by store and can be searched by the customer's phone number. Any old invoice can be opened and printed again. Staff only see the sales of their own store; the Admin sees all stores.

6. Inventory.

Store keepers add stock in, take stock out, or set the stock directly. Each change is written to the inventory history with the date, type and quantity, so the owner can always see why a number changed.

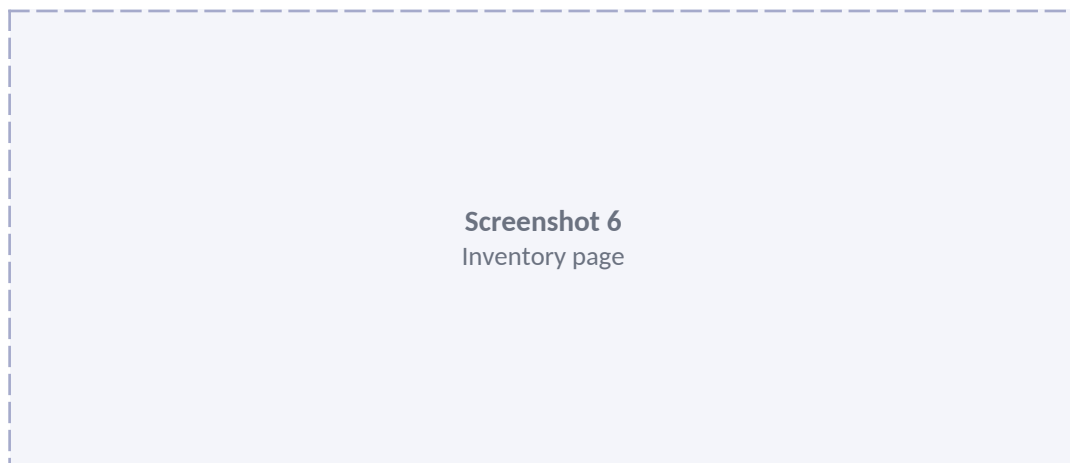


Figure 6: Inventory page

7. Damaged / spoiled goods.

Broken or expired items are recorded with a quantity and a reason. The stock is reduced automatically and the loss value appears on the dashboard and in the reports.

8. Multi-store management.

The Admin adds, edits and switches between stores and can view every store's data. A store that still has products, sales or staff cannot be deleted by mistake.

9. Workers management.

The Admin adds staff, sets their monthly salary, and can change a worker's role or store without creating a new account. The page shows each worker's face status, salary account and this month's attendance. If the same face is registered on two accounts, both accounts are flagged.

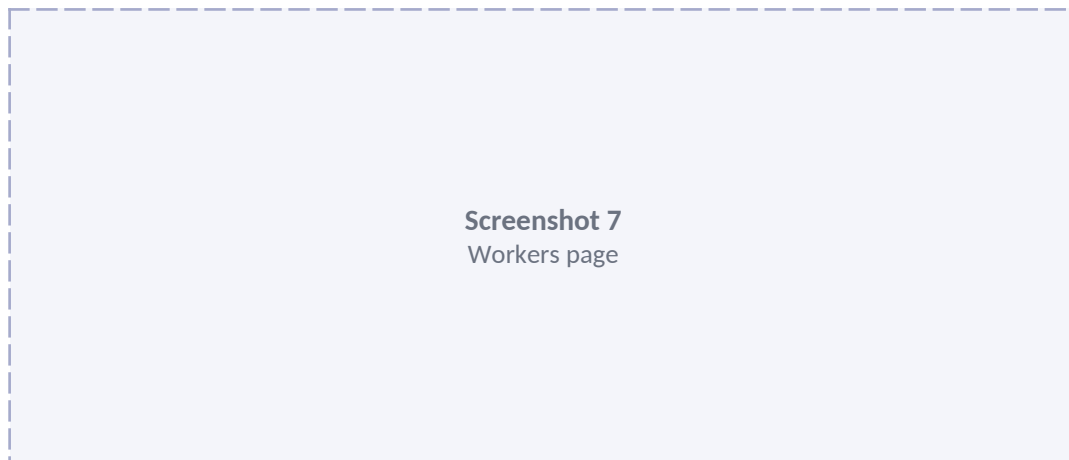


Figure 7: Workers page

10. Face-recognition attendance camera.

A staff member stands in front of the camera and the browser matches their face against the registered faces of that store. They are marked Present, or Late if they come after 9:15 AM. The "Close Day" button marks everyone who did not come as Absent. Friday is the weekly holiday, so nobody is marked absent on Friday.

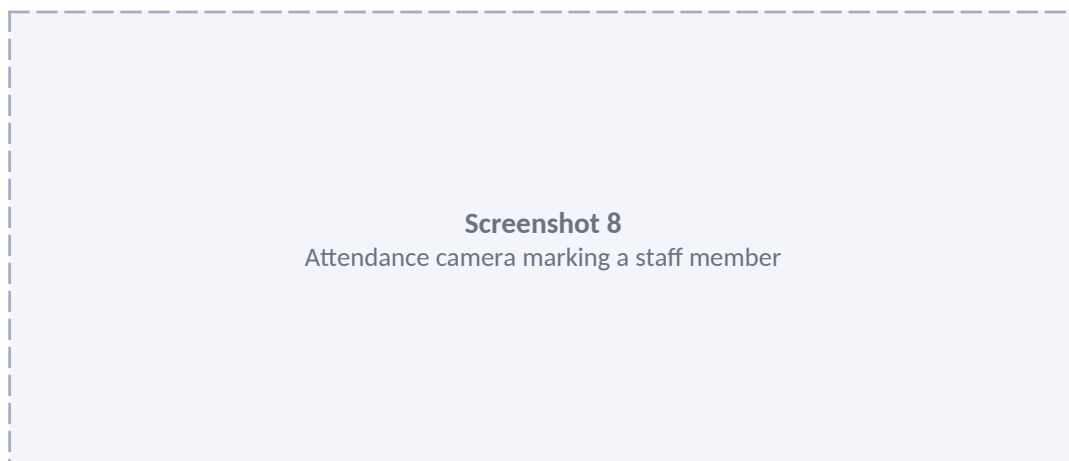


Figure 8: Attendance camera marking a staff member

11. Attendance and monthly summary.

Managers can also mark attendance by hand (Present, Late, Absent or Leave), but never for themselves. The monthly summary shows each worker's days, and a month's attendance is locked once its salary has been paid, so it cannot be changed afterwards.

12. Salary.

Salary is calculated from attendance. One day is worth $\text{salary} \div 30$. The first 3 absents and the first 4 lates in a month are free; after that each absent cuts one day and every 3 lates cut one day. A worker with no Present or Late day in the whole month gets nothing. A month can only be paid after it ends (from the 1st of the next month). The payment goes to the worker's chosen account (bank and mobile wallets are recorded as demo transfers) and is also saved as an expense. Staff can request to change their account, and the Admin approves it.

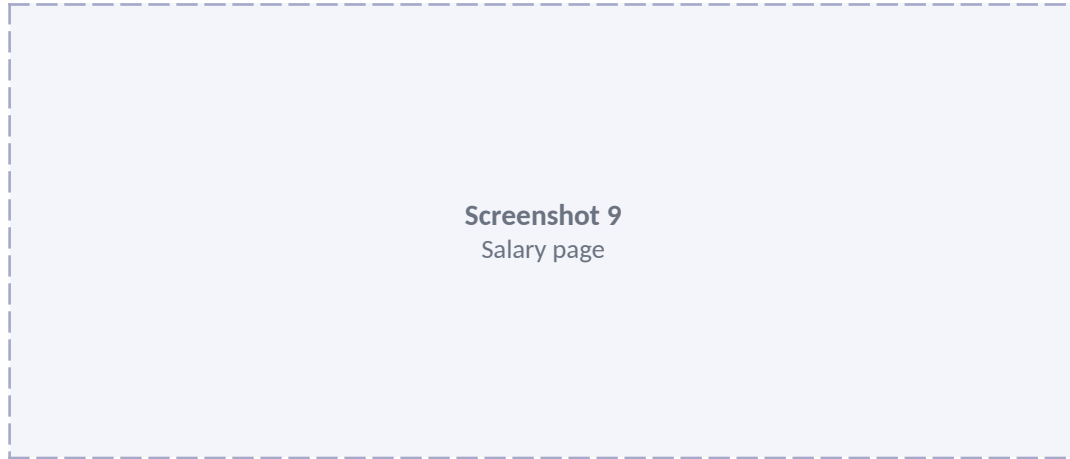


Figure 9: Salary page

13. Expenses and suppliers.

Each store records rent, bills and other expenses. Suppliers have a running due balance: a new due raises it, a payment lowers it, and the full ledger is kept.

14. Reports.

Monthly reports show total sales, number of orders, the payment method split, damaged-goods loss and the net amount for each store.

15. AI Insights and AI chat.

Admins and Managers get an AI assistant that answers questions such as "what sold most this week?" from live data. The same page shows anomaly alerts, a daily report and a simple sales forecast.

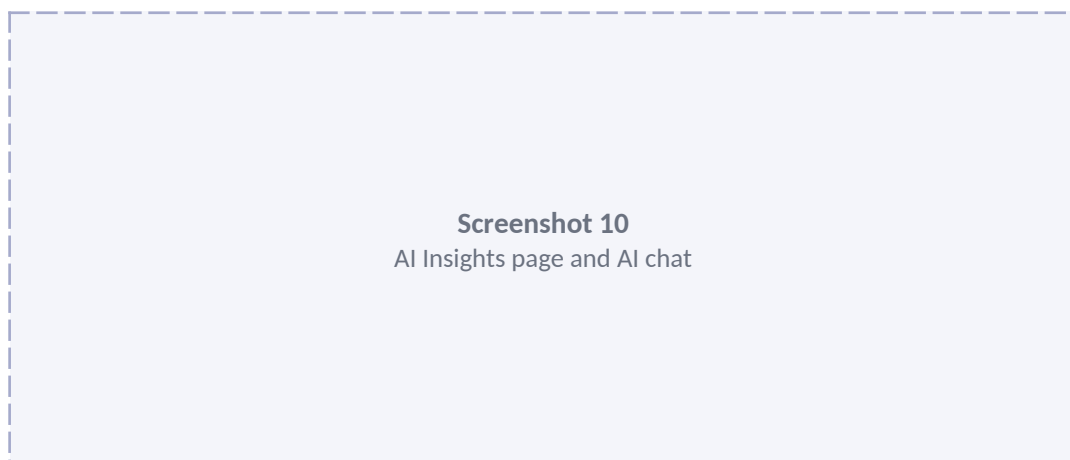


Figure 10: AI Insights page and AI chat

16. Customer portal.

Customers register with their phone number. They can search a product by name and see which store sells it cheapest, browse discounted offers, open a store's product list, see their purchases and loyalty points history, and chat with a customer AI assistant. A customer earns 1 point for every Tk 100 spent, and 100 points give a Tk 80 discount at checkout. Cost price, barcode and exact stock are never shown to customers.

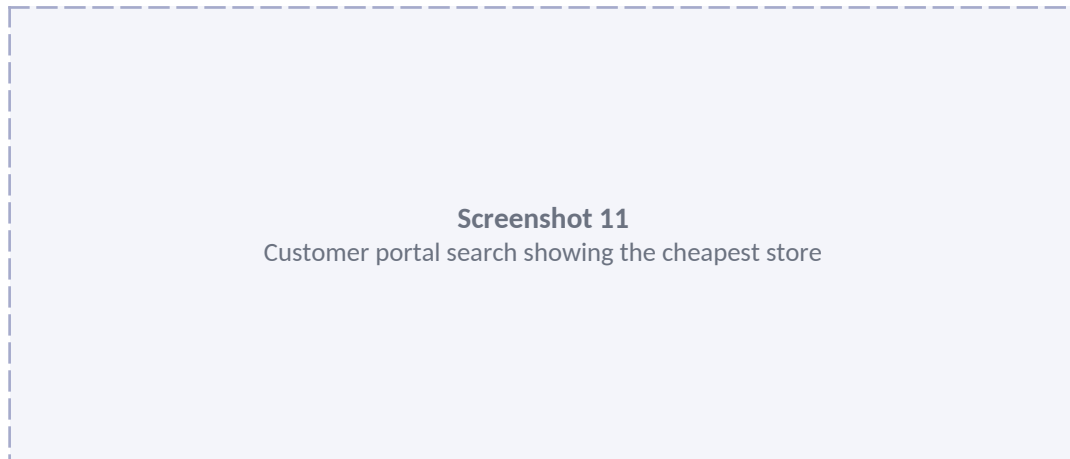


Figure 11: Customer portal search showing the cheapest store

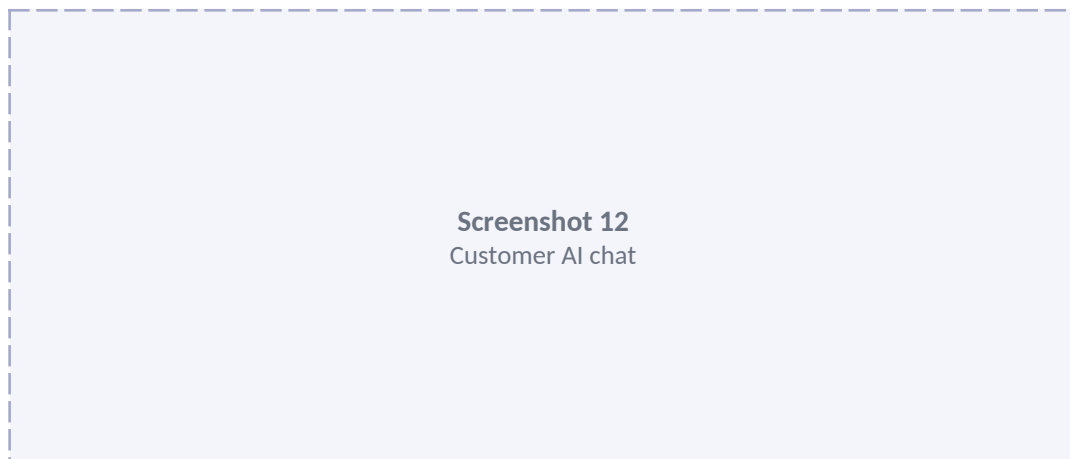


Figure 12: Customer AI chat

3. Technologies Used

We kept the stack simple on purpose: JavaScript on both sides, a normal SQL database, and free-tier cloud hosting that a small shop could actually afford. The system is split into three parts that talk to each other over a REST API.

3.1 System Architecture

Browser, server and cloud services talk over one REST API

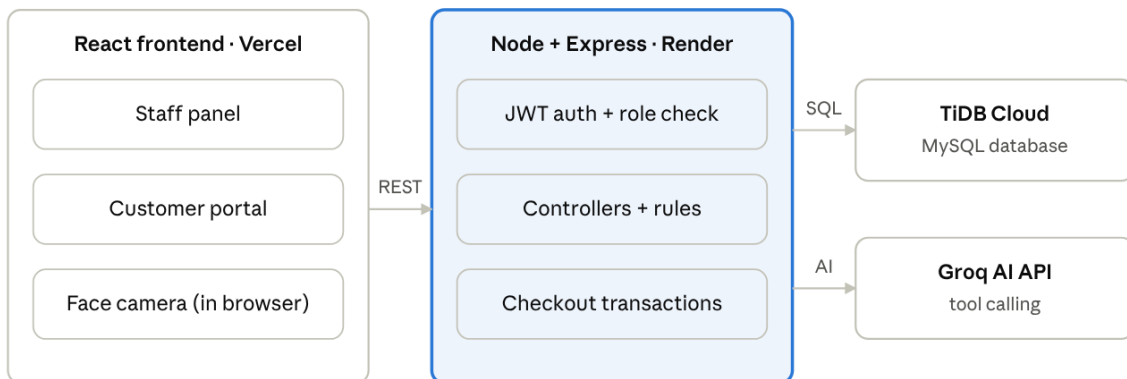


Figure 13: System architecture – frontend, backend, database and AI service

The browser only talks to our Express server, over HTTPS, with a JWT token. Only the server talks to the database and to the AI service, so database passwords, API keys and business rules never reach the browser.

3.2 Technology Stack

Area	What we used	How we used it
Frontend	React 19 (Create React App), Axios, plain CSS	One page per module (POS, Products, Salary and so on). Axios sends the login token with every request. We wrote our own CSS theme, SVG icons and toast notifications instead of using a UI library.
Backend	Node.js, Express 5	REST API split into routes and controllers (sales, products, salary, attendance, face, customers, AI...). Prices, stock and salary cuts are always checked again on the server.
Database	MySQL / MariaDB locally (XAMPP), TiDB Cloud in production	Tables for stores, users, products, sales, sale items, inventory history, customers, points, attendance, salary, expenses and suppliers. Schema changes are kept as numbered SQL migration files (001, 002, 003). Checkout runs inside a transaction with row locks so two cashiers cannot sell the same last item.

Area	What we used	How we used it
Authentication	JWT + bcrypt	Passwords are hashed with bcrypt. Login returns a JWT; every protected route checks it and then re-reads the user's role, store and status from the database, so a role change or a disabled account takes effect at once. Customer tokens cannot open staff APIs.
Face recognition	@vladmandic/human (runs in the browser)	On first login the camera captures a face embedding (a list of numbers, not a photo) and saves it. The attendance camera compares live faces with cosine similarity (match ≥ 0.6). The same face cannot be registered on two accounts.
AI models	Groq API (OpenAI-compatible) with tool calling	The AI does not guess numbers: it calls our own functions (search products, get offers, get sales, get points...) and answers from real data. Staff AI and customer AI use different tools, and the customer AI never sees cost price or barcode. There is a daily chat limit and a keyword fallback if the AI service is down.
APIs	Own REST API (around 60 endpoints)	Grouped as /api/auth, /api/products, /api/sales, /api/inventory-history, /api/damaged, /api/users, /api/stores, /api/attendance, /api/salary, /api/payout, /api/expenses, /api/suppliers, /api/reports, /api/face, /api/ai and /api/customers.
Barcode	@zxing/library, html5-qrcode	Camera and phone barcode scanning on the POS page.
Testing	Selenium WebDriver, pytest, pytest-html	8 automated end-to-end test cases (see Section 3.3).
Version control	Git and GitHub	Feature branches and pull requests (see Section 3.4).
Deployment	Vercel, Render, TiDB Cloud	Frontend, backend and database hosting (see Section 3.4).

Table 2: Technologies used in each part of the system

3.3 Testing

Besides testing by hand after every change, we wrote automated end-to-end tests with Selenium WebDriver and pytest. Each run opens Chrome, clicks through the app like a real user, and pytest-html saves an HTML report with a screenshot of every step. The tests are grouped into three features: login, products and admin pages.

ID	Feature	Test	Expected result
TC-01	Login	Admin logs in with the correct password	Control panel opens, badge shows Admin
TC-02	Login	Login with a wrong password	Error message, panel not shown
TC-03	Login	Logout	Back to the login form

ID	Feature	Test	Expected result
TC-04	Products	Add a new product	Success message, product appears in the list
TC-05	Products	Search the product list	Matching product is shown
TC-06	Admin pages	Admin menu	Dashboard, Products, Workers, Salary, Stores, Customers and Settings visible
TC-07	Admin pages	Open Workers page	Staff list loads with no error
TC-08	Admin pages	Open Salary page	Page loads without a database error

Table 3: Selenium test cases

All 8 tests passed in our last run. The tests also caught a real mistake for us: TC-08 failed once because an information message on the Salary page was styled like an error message. We fixed the style and the test passed again.

3.4 Version Control and Deployment

- **Git and GitHub.** The source code is on GitHub (github.com/meherabjim/Smart-Cloud-POS). We worked on feature branches (for example customer-portal-ai and pos-fixes) and merged them into main through pull requests, so a half-finished feature never broke the live site.
- **Keeping secrets out of Git.** The .env files, the test password file and the uploaded staff photos are listed in .gitignore, so they are never pushed.
- **Deployment.** The React frontend is hosted on Vercel, the Express backend on Render (smart-cloud-pos.onrender.com) and the database on TiDB Cloud. Pushing to main redeploys the site automatically. Environment variables on each host hold the database URL, the JWT secret and the AI key.

4. Future Works

4.1 Problems We Faced and How We Solved Them

Most of our time did not go into building new pages. It went into bugs that only showed up once the parts were joined together and tested with different roles. Table 4 lists the ones that taught us the most.

#	Problem we faced	How we solved it
1	A customer's login token could open staff APIs, because staff and customer tokens were signed with the same secret.	We added a check in the auth middleware: a customer token, or a token without a staff role, is rejected on every staff route.
2	When the Admin changed a worker from Store Keeper to Manager, the old token kept the old role for a whole day.	The server now reads the role, store and status from the database on every request, so the change works on the next click.
3	The same person could register their face on two accounts and mark attendance twice.	Before saving a face we compare it with every registered face; a match of 0.6 or more is refused, and old duplicates are flagged on the Workers page.
4	The POS trusted the browser: the discount could be over 100%, a cashier could sell from another store, and short cash was accepted.	Discount and tax are now limited to 0–100% and recalculated on the server, staff can only sell from their own store, and short cash is blocked on both sides.
5	Sales history could be opened without logging in.	All sales routes now need a token, and staff only see their own store's sales.
6	Salary could be paid for a month that was not finished, and attendance could be changed after payment.	A month can only be paid from the 1st of the next month, and its attendance is locked once the salary is paid.
7	Dates were saved in UTC, so attendance taken before 6 AM went to the previous day.	We now use Bangladesh time (UTC+6) on the server and the local date in the browser.
8	The local MySQL server timed out and phpMyAdmin would not open; later, tables like staff_accounts were missing.	mysqld was simply not running. We started it from XAMPP and ran the missing migration files. On TiDB we used a safe script with IF NOT EXISTS so it can be run twice without breaking anything.
9	Our JWT secret was committed to the public GitHub repository early on.	We moved all secrets to .env files, added them to .gitignore and are changing the secret on Render. Test passwords now live in an untracked config file.
10	CSS from one page leaked into another (for example .summary-label on both Products and Inventory).	Shared class names are now scoped under each page's own wrapper class.

Table 4: Main problems and their solutions

4.2 Upgrades and New Features

The system already works for a real shop, but there is a lot we would like to add if we continue the project:

1. **Real payments.** Connect bKash, Nagad and a card gateway so salary and customer payments move real money instead of being recorded as demo transfers.
2. **Offline POS mode.** Let the cashier keep selling when the internet drops and sync the sales when it comes back.
3. **Mobile app.** A small app for customers (points, offers, AI chat) and for staff attendance.
4. **Better AI.** Re-order suggestions for low stock, sales forecasts per product and a voice assistant in Bangla.
5. **Stronger face check.** A liveness test so a printed photo or a phone screen cannot be used at the attendance camera.
6. **More reports and dark mode.** Profit-and-loss per store, a VAT report, and export to Excel and PDF.
7. **More automated tests.** API tests for checkout and salary rules, run on every push with GitHub Actions.

5. Conclusion

Smart Cloud POS started as a simple billing screen and grew into a full shop system: sales, stock, several stores, staff attendance with face recognition, salary, customer loyalty and two AI assistants. It runs online on Vercel, Render and TiDB Cloud, and all 8 of our Selenium tests pass.

The biggest lesson for us was that a feature is not finished when the page looks right. It is finished when the server checks it too, when the wrong user cannot reach it, and when a test proves it still works after the next change. We plan to keep improving the project, starting with real payment gateways and an offline mode.

Appendix A: Running the Project Locally

These are the steps we follow to run the project on a new computer:

1. Install Node.js (v18 or newer), XAMPP (for MySQL) and Git.
2. Clone the repository: `git clone https://github.com/meherabjim/Smart-Cloud-POS.git`
3. Start MySQL from XAMPP, create the database and run the SQL files in backend/migrations in order (001, 002, 003).
4. In the backend folder, create a .env file with the database details, JWT_SECRET and GROQ_API_KEY, then run `npm install` and `npm start` (the server runs on port 5000).
5. In the frontend folder, run `npm install` and `npm start` (the app opens on port 3000).
6. To run the tests: install Python, then `pip install selenium pytest pytest-html`, and run `pytest --html=report.html` inside the tests folder.